NSF_Funding_Analysis_clean

June 2, 2026

1 NSF Funding Analysis (2021–2025)

DS 5500 — Chenjie Gu

This notebook analyzes NSF award data from 2021 to 2025, covering: 1. Funding trends over time by program/CFDA category 2. Top-funded institutions and states per year 3. Program Officer award counts 4. Topic modeling on award abstracts using LDA and BERTopic

> **Run cells in order top-to-bottom.** Each section depends on variables defined above it.
> All plots are saved as PNG files and then compiled into a single PDF at the end.

1.1 1. Setup & Installations

[1]:
```
import sys
!{sys.executable} -m pip install -q "urllib3<2" seaborn plotly pandas numpy␣
 ↪matplotlib openpyxl geopandas
!{sys.executable} -m pip install -q nltk gensim pyLDAvis
!{sys.executable} -m pip install -q bertopic sentence-transformers umap-learn␣
 ↪hdbscan scikit-learn
!{sys.executable} -m pip install -q pypdf Pillow kaleido

print("All packages installed.")
```

```
All packages installed.
```

[2]:
```
import os, re, ast, warnings, logging
from collections import Counter

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')          # non-interactive backend - avoids display errors
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import matplotlib.colors as mcolors
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
```

```
from plotly.subplots import make_subplots

import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
nltk.download('stopwords', quiet=True)
nltk.download('punkt',     quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('wordnet',   quiet=True)

import gensim
from gensim import corpora
from gensim.models import LdaModel, CoherenceModel
import pyLDAvis
import pyLDAvis.gensim_models

from bertopic import BERTopic
from sentence_transformers import SentenceTransformer
from umap import UMAP
from hdbscan import HDBSCAN
from sklearn.feature_extraction.text import CountVectorizer

warnings.filterwarnings('ignore')
logging.getLogger('gensim').setLevel(logging.ERROR)
logging.getLogger('bertopic').setLevel(logging.ERROR)


plt.rcParams['figure.figsize'] = (12, 6)
sns.set_theme(style='whitegrid')

#   Output folder for saved plots
PLOT_DIR = 'plots'
os.makedirs(PLOT_DIR, exist_ok=True)
SAVED_PLOTS = []   # running list - populated by save_fig() below

def save_fig(fig_or_name, name):
    """Save a matplotlib Figure or a Plotly figure to PLOT_DIR and register it.
    """
    path = os.path.join(PLOT_DIR, f'{name}.png')
    if hasattr(fig_or_name, 'write_image'):           # Plotly
        fig_or_name.write_image(path, width=1200, height=600, scale=2)
    elif hasattr(fig_or_name, 'savefig'):              # Matplotlib Figure
        fig_or_name.savefig(path, dpi=150, bbox_inches='tight')
        plt.close(fig_or_name)
    SAVED_PLOTS.append(path)
    print(f'  Saved → {path}')
```

```
print("All imports successful.")
print(f"Plots will be saved to: {os.path.abspath(PLOT_DIR)}")
```

```
All imports successful.
Plots will be saved to: /Users/_fin.fish_/Desktop/plots
```

1.2 2. Load & Inspect Data

[3]:
```
df = pd.read_excel('NSF_21_25_SG.xlsx', engine='openpyxl')
print(f'Loaded {len(df):,} rows')
print(df.dtypes.head(20))
```

```
Loaded 6,477 rows
Unnamed: 0.2                  int64
Unnamed: 0.1                  int64
Unnamed: 0                    int64
awd_id                        int64
agcy_id                      object
tran_type                    object
awd_istr_txt                 object
awd_titl_txt                 object
cfda_num                     object
org_code                      int64
po_phone                    float64
po_email                     object
po_sign_block_name           object
awd_eff_date         datetime64[ns]
awd_exp_date         datetime64[ns]
tot_intn_awd_amt              int64
awd_amount                    int64
awd_min_amd_letter_date  datetime64[ns]
awd_max_amd_letter_date  datetime64[ns]
abstract                     object
dtype: object
```

[4]:
```
#   Data Cleaning

df['awd_eff_date']   = pd.to_datetime(df['awd_eff_date'],  errors='coerce')
df['awd_exp_date']   = pd.to_datetime(df['awd_exp_date'],  errors='coerce')
df['year']           = df['awd_eff_year'].astype(int)
df['amount']         = pd.to_numeric(df['awd_amount'],      errors='coerce')
df['duration_years'] = ((df['awd_exp_date'] - df['awd_eff_date']).dt.days / 365.
 ↪25).round(2)
df['institution']    = df['inst.inst_name']
df['state']          = df['inst.inst_state_code']
df['country']        = df['inst.inst_country_name']
```

```
#   Extract PI name from nested JSON-like string
def extract_pi_name(pi_str):
    try:
        cleaned = str(pi_str).replace('None', '"None"')
        records = ast.literal_eval(cleaned)
        for r in records:
            if r.get('pi_role') == 'Principal Investigator':
                return r.get('pi_full_name', '')
        return records[0].get('pi_full_name', '') if records else ''
    except:
        return ''

#   Extract max obligation amount
def extract_oblg(oblg_str):
    try:
        records = ast.literal_eval(str(oblg_str))
        if records:
            return max(r.get('fund_oblg_amt', 0) for r in records)
    except:
        pass
    return np.nan

df['pi_name']      = df['pi'].apply(extract_pi_name)
df['oblg_amt']     = df['oblg_fy'].apply(extract_oblg)
df['cfda_primary'] = df['cfda_num'].astype(str).str.split(',').str[0].str.
 ↪strip()

#   Filter to 2021-2025
df = df[df['year'].between(2021, 2025)].copy()

print(f'Years: {df.year.min()} - {df.year.max()}')
print(f'Filtered to {len(df):,} awards (2021-2025)')
print(f'Missing amounts: {df.amount.isna().sum()}')
print(df.year.value_counts().sort_index())
```

```
Years: 2021 - 2025
Filtered to 6,398 awards (2021-2025)
Missing amounts: 0
year
2021    1278
2022    1441
2023    1462
2024    1194
2025    1023
Name: count, dtype: int64
```

1.3 3. Funding Trends Over Time

[5]:
```
#   Total NSF funding per year
yearly = df.groupby('year')['amount'].sum().reset_index()
yearly.columns = ['Year', 'Total Funding ($)']

fig = px.bar(
    yearly, x='Year', y='Total Funding ($)',
    title='Total NSF Funding per Year (2021-2025)',
    text_auto='.2s', color='Total Funding ($)',
    color_continuous_scale='Blues'
)
fig.update_layout(showlegend=False)
save_fig(fig, '01_total_funding_per_year')
fig.show()
```

```
Saved → plots/01_total_funding_per_year.png
```

[6]:
```
#   Funding by CFDA category
df_cfda = df.copy()
df_cfda['cfda_split'] = df_cfda['cfda_num'].astype(str).str.split(',')
df_cfda = df_cfda.explode('cfda_split')
df_cfda['cfda_split'] = df_cfda['cfda_split'].str.strip()
df_cfda['cfda_split'] = df_cfda['cfda_split'].replace('47.070', '47.07')

cfda_names = {
    '47.07':  'Computer & Info Science',
    '47.049': 'Math & Physical Sciences',
    '47.050': 'Geosciences',
    '47.074': 'Biological Sciences',
    '47.075': 'Social Sciences',
    '47.076': 'Education',
    '47.079': 'STEM Education',
    '47.083': 'Office of Integrative Activities',
    '47.041': 'Engineering',
}
df_cfda['cfda_label'] = df_cfda['cfda_split'].map(cfda_names).
 ↪fillna(df_cfda['cfda_split'])

cfda_year = df_cfda.groupby(['year', 'cfda_label'])['amount'].sum().
 ↪reset_index()
top_cfda = (
    cfda_year.groupby('cfda_label')['amount']
    .sum().nlargest(6).index.tolist()
)

fig = px.line(
    cfda_year[cfda_year['cfda_label'].isin(top_cfda)],
```

```
    x='year', y='amount', color='cfda_label',
    title='Top CFDA Categories by Funding Over Time',
    labels={'amount': 'Total Funding ($)', 'year': 'Year', 'cfda_label': 'CFDA␣
 ↪Category'},
    markers=True
)
save_fig(fig, '02_cfda_funding_trends')
fig.show()
```

```
Saved → plots/02_cfda_funding_trends.png
```

[7]:
```
#   Non-CSE CFDA sub-plots
other_cats = cfda_year[cfda_year['cfda_label'] != 'Computer & Info Science']
cats = other_cats['cfda_label'].unique()

fig = make_subplots(
    rows=2, cols=3,
    subplot_titles=list(cats[:6]),
    shared_yaxes=False
)
for i, cat in enumerate(cats[:6]):
    row, col = i // 3 + 1, i % 3 + 1
    data = other_cats[other_cats['cfda_label'] == cat]
    fig.add_trace(
        go.Bar(x=data['year'], y=data['amount'], name=cat, showlegend=False),
        row=row, col=col
    )
fig.update_layout(title='Non-CSE CFDA Categories by Funding Over Time',␣
 ↪height=600)
save_fig(fig, '03_non_cse_cfda_subplots')
fig.show()
```

```
Saved → plots/03_non_cse_cfda_subplots.png
```

[8]:
```
#   Treemap: total funding by CFDA
cfda_total = cfda_year.groupby('cfda_label')['amount'].sum().reset_index()

fig = px.treemap(
    cfda_total, path=['cfda_label'], values='amount',
    title='Total NSF Funding by CFDA Category (2021-2025)',
    color='amount', color_continuous_scale='Blues',
    labels={'amount': 'Total Funding ($)'}
)
fig.update_traces(textinfo='label+value+percent root')
save_fig(fig, '04_cfda_treemap')
fig.show()
```

```
Saved → plots/04_cfda_treemap.png
```

[9]:
```
#   CSE division funding per year
print('Division value counts:')
print(df['div_abbr'].value_counts())

div_yearly = df.groupby(['year', 'div_abbr']).agg(
    total_funding=('amount', 'sum'),
    num_awards=('awd_id', 'count')
).reset_index()

div_names = {
    'CSE': 'Computer and Information Science and Engineering (CISE)',
    'OAC': 'Advanced Cyberinfrastructure (CISE/OAC)',
    'CNS': 'Computer and Network Systems (CISE/CNS)',
    'CCF': 'Computing and Communication Foundations (CISE/CCF)',
    'IIS': 'Information and Intelligent Systems (CISE/IIS)',
}
div_yearly['division'] = div_yearly['div_abbr'].map(div_names).
 ↪fillna(div_yearly['div_abbr'])

fig1 = px.bar(
    div_yearly, x='year', y='total_funding', color='division',
    barmode='group',
    title='NSF CSE Funding by Division per Year',
    labels={'total_funding': 'Total Funding ($)', 'year': 'Year', 'division':␣
 ↪'Division'}
)
fig1.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
save_fig(fig1, '05_div_funding_per_year')
fig1.show()

fig2 = px.bar(
    div_yearly, x='year', y='num_awards', color='division',
    barmode='group',
    title='NSF CSE Award Count by Division per Year',
    labels={'num_awards': 'Number of Awards', 'year': 'Year', 'division':␣
 ↪'Division'}
)
fig2.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
save_fig(fig2, '06_div_awards_per_year')
fig2.show()
```

```
Division value counts:
div_abbr
CNS    2166
IIS    1601
CCF    1445
OAC    1186
```

```
Name: count, dtype: int64
  Saved → plots/05_div_funding_per_year.png

  Saved → plots/06_div_awards_per_year.png
```

[10]:
```python
#   Division totals (all years combined)
div_total = (
    df.groupby('div_abbr').agg(
        total_funding=('amount', 'sum'),
        num_awards=('awd_id', 'count')
    ).reset_index()
    .sort_values('total_funding', ascending=False)
)
div_names2 = {
    'OAC': 'Advanced Cyberinfrastructure (OAC)',
    'CNS': 'Computer and Network Systems (CNS)',
    'CCF': 'Computing and Communication Foundations (CCF)',
    'IIS': 'Information and Intelligent Systems (IIS)',
}
div_total['division'] = div_total['div_abbr'].map(div_names2).
 ↪fillna(div_total['div_abbr'])

fig1 = px.bar(
    div_total, x='division', y='total_funding',
    title='NSF CSE Total Funding by Division (2021-2025)',
    labels={'total_funding': 'Total Funding ($)', 'division': 'Division'},
    color='total_funding', color_continuous_scale='Blues', height=500
)
fig1.update_layout(xaxis_tickangle=-15)
save_fig(fig1, '07_div_total_funding')
fig1.show()

fig2 = px.bar(
    div_total, x='division', y='num_awards',
    title='NSF CSE Total Awards by Division (2021-2025)',
    labels={'num_awards': 'Number of Awards', 'division': 'Division'},
    color='num_awards', color_continuous_scale='Blues', height=500
)
fig2.update_layout(xaxis_tickangle=-15)
save_fig(fig2, '08_div_total_awards')
fig2.show()

fig4 = px.pie(
    div_total, names='division', values='total_funding',
    title='NSF CSE Funding Share by Division (2021-2025)',
    color_discrete_sequence=px.colors.sequential.Blues_r
)
fig4.update_traces(textposition='inside', textinfo='percent+label')
```

```
save_fig(fig4, '09_div_funding_pie')
fig4.show()
```

```
Saved → plots/07_div_total_funding.png
Saved → plots/08_div_total_awards.png
Saved → plots/09_div_funding_pie.png
```

1.4 4. Institutional & Geographic Analysis

[11]:
```
#   Institution totals
inst_total = (
    df.groupby('institution')['amount']
    .sum().reset_index()
    .sort_values('amount', ascending=False)
)

fig1 = px.bar(
    inst_total, x='institution', y='amount',
    title='Total NSF CSE Funding by Institution (2021-2025)',
    labels={'amount': 'Total Funding ($)', 'institution': 'Institution'},
    color='amount', color_continuous_scale='Blues', height=600
)
fig1.update_layout(xaxis_tickangle=-90, xaxis_tickfont_size=7)
save_fig(fig1, '10_inst_total_bar')
fig1.show()

fig2 = px.treemap(
    inst_total, path=['institution'], values='amount',
    title='NSF CSE Funding by Institution - Treemap (2021-2025)',
    color='amount', color_continuous_scale='Blues',
)
fig2.update_traces(textinfo='label+value')
save_fig(fig2, '11_inst_treemap')
fig2.show()

inst_counts = (
    df.groupby('institution').agg(
        total_funding=('amount', 'sum'),
        num_awards=('awd_id', 'count')
    ).reset_index()
    .sort_values('total_funding', ascending=False)
    .head(50)
)
fig3 = px.scatter(
    inst_counts, x='num_awards', y='total_funding',
    size='total_funding', hover_name='institution',
```

```
    title='NSF CSE Institutions - Bubble Chart (Top 50, 2021-2025)',
    labels={'total_funding': 'Total Funding ($)', 'num_awards': 'Number of␣
 ↪Awards'},
    color='total_funding', color_continuous_scale='Teal', height=600
)
save_fig(fig3, '12_inst_bubble')
fig3.show()
```

```
Saved → plots/10_inst_total_bar.png
```

```
Saved → plots/11_inst_treemap.png
```

```
Saved → plots/12_inst_bubble.png
```

[12]:
```
#   State-level funding maps
import geopandas as gpd

state_year = (
    df.groupby(['year', 'state'])['amount']
    .sum().reset_index()
)
print('Top states by total funding:')
print(state_year.groupby('state')['amount'].sum().nlargest(10))

gdf = gpd.read_file(
    'https://raw.githubusercontent.com/nvkelso/natural-earth-vector/master/'
    'geojson/ne_110m_admin_1_states_provinces.geojson'
)
gdf = gdf[gdf['iso_a2'] == 'US'].copy()
gdf['state'] = gdf['postal'].str.strip()

vmin = state_year['amount'].min()
vmax = state_year['amount'].max()
years = sorted(state_year['year'].unique())

for yr in years:
    yr_data = state_year[state_year['year'] == yr]
    merged  = gdf.merge(yr_data, on='state', how='left')

    fig_map, ax = plt.subplots(1, 1, figsize=(14, 8))
    merged.plot(
        column='amount', ax=ax, cmap='YlOrRd',
        vmin=vmin, vmax=vmax,
        edgecolor='black', linewidth=0.3,
        legend=True,
        legend_kwds={
            'label': 'Total Funding ($)',
            'orientation': 'vertical',
```

```
            'shrink': 0.6,
            'format': lambda x, _: f'${x/1e6:.0f}M'
        },
        missing_kwds={'color': 'lightgrey', 'label': 'No Data'}
    )
    ax.set_title(f'NSF CSE Funding by State - {yr}', fontsize=16,␣
↪fontweight='bold')
    ax.axis('off')
    plt.tight_layout()
    save_fig(fig_map, f'13_state_map_{yr}')
    plt.show()
```

```
Top states by total funding:
state
CA    364532018
NY    214423772
TX    187578298
MA    176824420
IL    172946611
PA    152805549
VA    109501412
IN    100800849
NJ     92184105
MI     90980418
Name: amount, dtype: int64
  Saved → plots/13_state_map_2021.png
  Saved → plots/13_state_map_2022.png
  Saved → plots/13_state_map_2023.png
  Saved → plots/13_state_map_2024.png
  Saved → plots/13_state_map_2025.png
```

1.5 5. Principal Investigator Analysis

[13]:
```
# pi_name already extracted during data cleaning; verify here
print('Top PI names:')
print(df['pi_name'].value_counts().head(10))
print(f'Empty pi_names: {(df["pi_name"] == "").sum()}')

pi_total = (
    df.groupby('pi_name')['awd_id']
    .count().reset_index()
    .rename(columns={'awd_id': 'num_awards'})
    .sort_values('num_awards', ascending=False)
)

fig1 = px.bar(
    pi_total, x='pi_name', y='num_awards',
```

```
    title='All Principal Investigators by Total Awards (2021-2025)',
    labels={'num_awards': 'Total Awards', 'pi_name': 'Principal Investigator'},
    color='num_awards', color_continuous_scale='turbo', height=600
)
fig1.update_layout(xaxis_tickangle=-90, xaxis_tickfont_size=6,
                   plot_bgcolor='white', paper_bgcolor='white')
save_fig(fig1, '14_pi_awards_bar')
fig1.show()

fig2 = px.treemap(
    pi_total, path=['pi_name'], values='num_awards',
    title='All Principal Investigators - Treemap (2021-2025)',
    color='num_awards', color_continuous_scale='Blues',
    labels={'num_awards': 'Total Awards'}
)
fig2.update_traces(textinfo='label+value')
save_fig(fig2, '15_pi_treemap')
fig2.show()

pi_bubble = (
    df.groupby('pi_name').agg(
        total_awards=('awd_id', 'count'),
        total_funding=('amount', 'sum')
    ).reset_index()
    .sort_values('total_awards', ascending=False)
)
fig3 = px.scatter(
    pi_bubble, x='total_awards', y='total_funding',
    size='total_awards', hover_name='pi_name',
    title='All Principal Investigators - Bubble Chart (2021-2025)',
    labels={'total_funding': 'Total Funding ($)', 'total_awards': 'Number of␣
 ↪Awards'},
    color='total_funding', color_continuous_scale='Blues', height=600
)
save_fig(fig3, '16_pi_bubble')
fig3.show()
```

```
Top PI names:
pi_name
Ewa Deelman            10
Marouane Kessentini    10
Yanfang Ye              9
Xin Liang               8
Ferdinando Fioretto     8
Deliang Fan             7
Josiah D Hester         7
Wenbin Zhang            7
Fatemeh Afghah          7
```

```
Wei Wang                6
Name: count, dtype: int64
Empty pi_names: 0
  Saved → plots/14_pi_awards_bar.png

  Saved → plots/15_pi_treemap.png

  Saved → plots/16_pi_bubble.png
```

[14]:
```python
#   Award Duration Distribution
df['duration_rounded'] = df['duration_years'].round(0)
avg_duration = df['duration_rounded'].dropna()

print(f'Mean award duration:   {avg_duration.mean():.2f} years')
print(f'Median award duration: {avg_duration.median():.2f} years')
print(df['duration_rounded'].value_counts().sort_index())

fig = px.histogram(
    df, x='duration_rounded',
    title='Distribution of NSF Award Durations',
    labels={'duration_rounded': 'Duration (Years)'},
    color_discrete_sequence=['steelblue'],
    nbins=7
)
fig.update_traces(xbins=dict(start=0, end=7, size=1))
fig.add_vline(x=avg_duration.mean(), line_dash='dash', line_color='red',
              annotation_text=f'Mean: {avg_duration.mean():.1f} yrs')
fig.update_layout(bargap=0.1)
save_fig(fig, '17_award_duration_hist')
fig.show()
```

```
Mean award duration:   2.91 years
Median award duration: 3.00 years
duration_rounded
0.0     170
1.0     683
2.0    1159
3.0    2382
4.0    1558
5.0     442
6.0       4
Name: count, dtype: int64
  Saved → plots/17_award_duration_hist.png
```

1.6 6. Topic Modeling on Award Abstracts

We apply two methods: - **LDA** — classical bag-of-words probabilistic model - **BERTopic** — transformer-based with BERT embeddings + UMAP + HDBSCAN

[15]:
```
#   Inspect abstract / corpus columns
print('=== abstract sample ===')
print(df['abstract'].iloc[0][:300])
print(f'abstract non-null: {df["abstract"].notna().sum()}')
print('\n=== corpus sample ===')
print(df['corpus'].iloc[0][:300])
print(f'corpus non-null: {df["corpus"].notna().sum()}')
same = (df['abstract'] == df['corpus']).sum()
print(f'\nRows where abstract == corpus: {same} out of {len(df)}')
```

```
=== abstract sample ===
particle and nuclear physics pnp are fundamentally probabilistic due to quantum
mechanics both fields rely on complex montecarlo mcbased simulators that use
random number sampling to make predictions for nearly all aspects of
experimental design and data interpretation in fact most branches of scien
abstract non-null: 6398

=== corpus sample ===
Collaborative Research: CyberTraining: Pilot: Monte Carlo General Education
Network (MCGEN)Particle and nuclear physics (PNP) are fundamentally
probabilistic due to quantum mechanics. Both fields rely on complex Monte-Carlo
(MC)-based simulators that use random number sampling to make predictions fo
corpus non-null: 6398

Rows where abstract == corpus: 0 out of 6398
```

[16]:
```
#   CS subfield keyword dictionary
cs_subfields = {
    'machine_learning': [
        'neural', 'network', 'deep', 'learning', 'training', 'model',
        'classification', 'prediction', 'reinforcement', 'supervised',
        'unsupervised', 'transformer', 'attention', 'embedding', 'gradient',
        'backpropagation', 'convolution', 'lstm', 'generative', 'adversarial',
        'diffusion', 'llm', 'language', 'foundation', 'finetuning', 'inference',
        'learn', 'train', 'machine', 'artificial', 'intelligence',
        'compute', 'automate', 'feature', 'representation', 'recognition',
        'detection', 'generation', 'evaluation', 'benchmark',
    ],
    'cybersecurity': [
        'security', 'privacy', 'attack', 'defense', 'vulnerability',
        'malware', 'encryption', 'authentication', 'intrusion', 'detection',
        'cryptography', 'adversarial', 'threat', 'firewall', 'forensic',
        'cyber', 'breach', 'exploit', 'protocol', 'certificate', 'trust',
        'identity', 'authorization', 'blockchain', 'secure', 'trustworthy',
    ],
    'networking': [
        'network', 'wireless', 'spectrum', 'bandwidth', 'latency',
        'protocol', 'routing', 'communication', 'internet', 'edge',
```

```
        'cloud', 'distributed', 'packet', 'topology', '5g', 'mimo',
        'antenna', 'channel', 'throughput', 'congestion', 'qos',
        'cellular', 'wifi', 'iot', 'sensor', 'service', 'access', 'device',
    ],
    'robotics': [
        'robot', 'autonomous', 'vehicle', 'drone', 'manipulation',
        'navigation', 'planning', 'perception', 'control', 'actuator',
        'locomotion', 'swarm', 'humanoid', 'collaborative', 'teleoperation',
        'haptic', 'grasping', 'mapping', 'slam', 'motion',
    ],
    'hci': [
        'human', 'interaction', 'interface', 'user', 'accessibility',
        'visualization', 'augmented', 'virtual', 'reality', 'wearable',
        'gesture', 'haptic', 'usability', 'experience', 'cognitive',
        'social', 'collaboration', 'crowdsourcing', 'gamification',
        'education', 'technology', 'understand', 'diverse', 'practice',
    ],
    'algorithms': [
        'algorithm', 'complexity', 'optimization', 'graph', 'combinatorial',
        'approximation', 'randomized', 'online', 'streaming', 'parallel',
        'distributed', 'computational', 'geometry', 'sorting', 'search',
        'dynamic', 'programming', 'heuristic', 'polynomial', 'nphard',
    ],
    'systems': [
        'operating', 'memory', 'storage', 'cache', 'processor', 'compiler',
        'runtime', 'kernel', 'virtualization', 'container', 'performance',
        'scalability', 'fault', 'tolerance', 'scheduling', 'concurrency',
        'parallelism', 'hardware', 'architecture', 'fpga', 'gpu', 'cpu',
        'computer', 'engineer', 'technology', 'device', 'test',
    ],
    'software_engineering': [
        'software', 'code', 'testing', 'debugging', 'verification',
        'specification', 'refactoring', 'maintenance', 'documentation',
        'agile', 'devops', 'deployment', 'bug', 'patch', 'repository',
        'version', 'static', 'analysis', 'formal', 'methods',
    ],
    'data_science': [
        'database', 'query', 'mining', 'analytics', 'warehouse',
        'streaming', 'knowledge', 'graph', 'ontology', 'semantic',
        'retrieval', 'indexing', 'clustering', 'anomaly', 'pattern',
        'statistical', 'bayesian', 'causal', 'inference', 'fairness',
    ],
    'quantum_computing': [
        'quantum', 'qubit', 'entanglement', 'superposition', 'circuit',
        'gate', 'decoherence', 'error', 'correction', 'supremacy',
        'annealing', 'cryptography', 'simulation', 'speedup',
    ],
```

```
    'cyberinfrastructure': [
        'cyberinfrastructure', 'hpc', 'supercomputing', 'cluster',
        'grid', 'workflow', 'pipeline', 'reproducibility', 'openscience',
        'fairdata', 'provenance', 'interoperability', 'metadata',
        'repository', 'portal', 'gateway', 'middleware',
    ],
    'health_ai': [
        'health', 'medical', 'clinical', 'patient', 'diagnosis',
        'treatment', 'disease', 'imaging', 'genomics', 'bioinformatics',
        'drug', 'electronic', 'record', 'hospital', 'wearable',
        'monitoring', 'epidemiology', 'precision', 'biomedical',
    ],
}

cs_keywords_keep = set(w for kws in cs_subfields.values() for w in kws)

#   Aggressive stop-word list
stop_words_cs = set(stopwords.words('english'))
stop_words_cs.update([
    'research', 'study', 'project', 'award', 'nsf', 'program',
    'support', 'develop', 'development', 'provide', 'also', 'use',
    'used', 'using', 'based', 'work', 'new', 'result', 'include',
    'will', 'may', 'approach', 'framework', 'tool', 'application',
    'technique', 'method', 'propose', 'proposed', 'present',
    'investigate', 'explore', 'examine', 'address', 'focus',
    'enable', 'allow', 'improve', 'increase', 'reduce',
    'important', 'critical', 'novel', 'innovative', 'effective',
    'efficient', 'robust', 'scalable', 'real', 'large', 'high',
    'wide', 'broad', 'multiple', 'various', 'different', 'existing',
    'current', 'future', 'potential', 'significant', 'key', 'main',
    'major', 'general', 'specific', 'particular', 'certain',
    'investigator', 'grant', 'funded', 'funding', 'university',
    'college', 'national', 'foundation', 'science', 'scientific',
    'researcher', 'student', 'team', 'faculty', 'professor',
    'graduate', 'undergraduate', 'phd', 'postdoc', 'broader',
    'intellectual', 'merit', 'impact', 'advance', 'goal', 'objective',
    'challenge', 'problem', 'solution', 'context', 'area', 'field',
    'domain', 'community', 'society', 'public', 'government',
    'industry', 'partner', 'collaboration', 'workshop', 'conference',
    'paper', 'publication', 'dataset', 'benchmark', 'experiment',
    'make', 'build', 'create', 'design', 'show', 'demonstrate',
    'achieve', 'obtain', 'perform', 'apply', 'extend', 'combine',
    'integrate', 'leverage', 'utilize', 'deploy', 'implement',
    'system', 'model', 'data', 'information', 'knowledge', 'task',
    'process', 'resource', 'environment', 'platform', 'infrastructure',
    'component', 'module', 'layer', 'level', 'step', 'stage',
    'aspect', 'feature', 'property', 'factor', 'element', 'type',
```

```
    'class', 'category', 'group', 'set', 'list', 'number', 'case',
    'example', 'instance', 'scenario', 'situation', 'condition',
    'mission', 'reflect', 'review', 'criterion', 'deem', 'statutory',
    'worthy', 'evaluation', 'outcome', 'activity', 'opportunity',
    'effort', 'practice', 'enhance', 'across', 'identify', 'require',
    'lead', 'share', 'access', 'plan', 'help', 'need', 'understand',
    'exist', 'however', 'well', 'many', 'first', 'three', 'time',
    'base', 'diverse', 'complex', 'service', 'material', 'state',
    'structure', 'thrust', 'benefit', 'distribute', 'change',
    'evaluate', 'involve', 'limit', 'range', 'institution', 'course',
    'capability', 'energy',
])

lemmatizer = WordNetLemmatizer()

def preprocess_cs(text):
    if not isinstance(text, str) or len(text) < 20:
        return []
    text = re.sub(r'<.*?>', ' ', text)
    text = re.sub(r'\d+', ' ', text)
    text = re.sub(r'[^a-zA-Z\s]', ' ', text)
    text = text.lower()
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t, pos='v') for t in tokens]
    tokens = [lemmatizer.lemmatize(t, pos='n') for t in tokens]
    tokens = [t for t in tokens
              if t not in stop_words_cs and 3 < len(t) < 25]
    return tokens

def cs_keyword_ratio(tokens):
    if not tokens:
        return 0
    return sum(1 for t in tokens if t in cs_keywords_keep) / len(tokens)

#   Prepare abstract and corpus dataframes
df_text_abstract = df[df['abstract'].notna()].copy()
df_text_abstract['tokens'] = df_text_abstract['abstract'].apply(preprocess_cs)
df_text_abstract = df_text_abstract[df_text_abstract['tokens'].apply(len) >=␣
 ↪10].copy()
df_text_abstract['cs_ratio'] = df_text_abstract['tokens'].
 ↪apply(cs_keyword_ratio)

df_text_corpus = df[df['corpus'].notna()].copy()
df_text_corpus['tokens'] = df_text_corpus['corpus'].apply(preprocess_cs)
df_text_corpus = df_text_corpus[df_text_corpus['tokens'].apply(len) >= 10].
 ↪copy()
df_text_corpus['cs_ratio'] = df_text_corpus['tokens'].apply(cs_keyword_ratio)
```

```
print(f'Abstract documents:  {len(df_text_abstract):,}')
print(f'Corpus documents:    {len(df_text_corpus):,}')
print(f'Avg CS ratio - Abstract: {df_text_abstract["cs_ratio"].mean():.2%}')
print(f'Avg CS ratio - Corpus:   {df_text_corpus["cs_ratio"].mean():.2%}')
```

```
Abstract documents:  6,398
Corpus documents:    6,398
Avg CS ratio - Abstract: 20.28%
Avg CS ratio - Corpus:   21.74%
```

1.6.1 6a. LDA Topic Modeling

[17]:
```
def build_gensim_corpus(tokens):
    """Build gensim dictionary and BoW corpus from a list of token lists."""
    dictionary = corpora.Dictionary(tokens)
    dictionary.filter_extremes(no_below=15, no_above=0.4)
    corpus = [dictionary.doc2bow(t) for t in tokens]
    return dictionary, corpus

def find_best_k(tokens, label, k_range=range(5, 35, 5)):
    dictionary, corpus = build_gensim_corpus(tokens)
    coherence_scores = []
    for k in k_range:
        lda = LdaModel(corpus=corpus, id2word=dictionary,
                       num_topics=k, random_state=42,
                       passes=15, iterations=150,
                       alpha='auto', eta='auto')
        cm = CoherenceModel(model=lda, texts=tokens,
                            dictionary=dictionary, coherence='c_v')
        score = cm.get_coherence()
        coherence_scores.append(score)
        print(f'  [{label}] k={k:2d}: coherence = {score:.4f}')
    best_k = list(k_range)[coherence_scores.index(max(coherence_scores))]
    print(f'\n  [{label}] Best k = {best_k} (coherence = {max(coherence_scores):
 ↪.4f})\n')
    return best_k, dictionary, corpus, coherence_scores, list(k_range)

print('=== Abstract ===')
best_k_abstract, dictionary_abs, corpus_abs, scores_abs, k_range_abs =␣
 ↪find_best_k(
    df_text_abstract['tokens'].tolist(), 'abstract'
)

print('=== Corpus ===')
best_k_corpus, dictionary_corp, corpus_corp, scores_corp, k_range_corp =␣
 ↪find_best_k(
```

```
    df_text_corpus['tokens'].tolist(), 'corpus'
)

fig_coh, ax_coh = plt.subplots(figsize=(10, 4))
ax_coh.plot(k_range_abs,  scores_abs,  marker='o', label='Abstract',␣
 ↪color='steelblue')
ax_coh.plot(k_range_corp, scores_corp, marker='s', label='Corpus',  ␣
 ↪color='coral')
ax_coh.set_title('LDA Coherence Score by Number of Topics')
ax_coh.set_xlabel('Number of Topics (k)')
ax_coh.set_ylabel('Coherence Score (c_v)')
ax_coh.legend()
plt.tight_layout()
save_fig(fig_coh, '18_lda_coherence')
plt.show()

print(f'Best k - Abstract: {best_k_abstract} | Corpus: {best_k_corpus}')
```

```
=== Abstract ===
  [abstract] k= 5: coherence = 0.4157
  [abstract] k=10: coherence = 0.4782
  [abstract] k=15: coherence = 0.4869
  [abstract] k=20: coherence = 0.5010
  [abstract] k=25: coherence = 0.4957
  [abstract] k=30: coherence = 0.4975

  [abstract] Best k = 20 (coherence = 0.5010)

=== Corpus ===
  [corpus] k= 5: coherence = 0.4195
  [corpus] k=10: coherence = 0.4954
  [corpus] k=15: coherence = 0.4875
  [corpus] k=20: coherence = 0.4876
  [corpus] k=25: coherence = 0.4971
  [corpus] k=30: coherence = 0.4697

  [corpus] Best k = 25 (coherence = 0.4971)

  Saved → plots/18_lda_coherence.png
Best k - Abstract: 20 | Corpus: 25
```

[18]:
```
#   Train final LDA models (k=15 for comparability)
best_k_abstract = 15
best_k_corpus   = 15
print('Using k=15 for both models')

lda_abstract = LdaModel(
```

```
    corpus=corpus_abs, id2word=dictionary_abs,
    num_topics=best_k_abstract, random_state=42,
    passes=40, iterations=400,
    alpha='auto', eta='auto',
    chunksize=2000, minimum_probability=0.01
)
print('  LDA Abstract Topics  \n')
for idx, topic in lda_abstract.print_topics(num_words=10):
    words = [w.split('*')[1].replace('"','').strip() for w in topic.split('+')]
    print(f'Topic {idx+1:2d}: {" | ".join(words)}')

lda_corpus = LdaModel(
    corpus=corpus_corp, id2word=dictionary_corp,
    num_topics=best_k_corpus, random_state=42,
    passes=40, iterations=400,
    alpha='auto', eta='auto',
    chunksize=2000, minimum_probability=0.01
)
print('\n  LDA Corpus Topics  \n')
for idx, topic in lda_corpus.print_topics(num_words=10):
    words = [w.split('*')[1].replace('"','').strip() for w in topic.split('+')]
    print(f'Topic {idx+1:2d}: {" | ".join(words)}')
```

```
Using k=15 for both models
  LDA Abstract Topics

Topic  1: network | communication | wireless | device | edge | internet |
spectrum | performance | sense | control
Topic  2: health | social | human | technology | people | interaction |
individual | patient | intervention | disease
Topic  3: language | test | verification | reason | code | software | agent |
generate | formal | automate
Topic  4: compute | quantum | hardware | performance | software | memory |
computer | architecture | technology | simulation
Topic  5: user | device | mobile | security | protocol | authentication | video
| cloud | content | accessibility
Topic  6: software | policy | user | analysis | technology | engineer |
repository | online | legal | visualization
Topic  7: security | attack | detection | vulnerability | cybersecurity | threat
| detect | defense | secure | adversarial
Topic  8: disaster | resilience | city | management | event | technology | water
| response | local | mobility
Topic  9: algorithm | network | graph | neural | machine | computational |
theory | deep | optimization | train
Topic 10: simulation | engineer | vehicle | computational | power | dynamic |
safety | physical | manufacture | autonomous
Topic 11: experience | site | participant | network | mentor | career | computer
| technology | participate | security
```

```
Topic 12: privacy | datasets | machine | query | search | analysis | database |
user | storage | compression
Topic 13: decision | fairness | bias | decisionmaking | fair | machine |
algorithmic | analytics | issue | intelligence
Topic 14: train | education | compute | teacher | school | workforce | computer
| skill | cyberinfrastructure | stem
Topic 15: robot | human | image | object | vision | computer | control |
robotics | visual | physical

  LDA Corpus Topics

Topic  1: network | device | wireless | communication | edge | power | sense |
spectrum | sensor | mobile
Topic  2: education | compute | teacher | school | computer | stem | teach |
campus | educational | skill
Topic  3: social | decision | online | fairness | individual | bias | medium |
risk | intervention | policy
Topic  4: site | experience | computer | compute | career | mentor | participant
| travel | participation | underrepresented
Topic  5: human | user | robot | language | people | interaction | computer |
technology | visual | speech
Topic  6: algorithm | quantum | theory | optimization | complexity | machine |
computational | theoretical | small | error
Topic  7: privacy | security | attack | secure | user | satc | vulnerability |
threat | core | cybersecurity
Topic  8: network | graph | machine | neural | drive | autonomous | safety |
control | algorithm | deep
Topic  9: analysis | computational | cluster | storage | visualization | machine
| discovery | scale | datasets | cell
Topic 10: disaster | event | resilience | management | internet | water |
response | network | monitor | remote
Topic 11: software | open | source | simulation | engineer | user |
cyberinfrastructure | scale | test | code
Topic 12: image | computational | object | space | computer | physic | shape |
surface | representation | flow
Topic 13: health | technology | patient | workforce | intelligence | artificial
| engineer | medical | center | healthcare
Topic 14: city | smart | mobility | transportation | civic | food | house |
vehicle | technology | urban
Topic 15: compute | performance | software | hardware | memory | architecture |
cloud | computer | scale | code
```

[19]:
```python
#   Assign dominant topic & CS subfield labels
def get_dominant_topic(bow, model):
    topics = model.get_document_topics(bow)
    return max(topics, key=lambda x: x[1])[0] if topics else -1
```

```
abstract_topic_labels = {
    0:  'STEM Education & Workforce',
    1:  'Health & Biomedical AI',
    2:  'Algorithms, ML & Quantum',
    3:  'Computer Vision & Robotics',
    4:  'Cyberinfrastructure & HPC',
    5:  'Autonomous Systems & Transportation',
    6:  'Software Engineering & Formal Methods',
    7:  'Cybersecurity & Privacy',
    8:  'Networking & Wireless',
    9:  'K-12 CS Education',
    10: 'Academic Community & Outreach',
    11: 'Data Science & Open Source',
    12: 'Human-Computer Interaction',
    13: 'Scientific Computing & Simulation',
    14: 'Systems & Architecture',
}
corpus_topic_labels = {
    0:  'Academic Community & Outreach',
    1:  'Algorithms, Quantum & Theory',
    2:  'Robotics & Autonomous Systems',
    3:  'Cybersecurity',
    4:  'Privacy, Fairness & Social AI',
    5:  'Wireless & Networking',
    6:  'Network Systems & Databases',
    7:  'CS Education & STEM',
    8:  'Health, Disaster & Resilience',
    9:  'Systems, Hardware & Edge',
    10: 'Human-Computer Interaction',
    11: 'Software Engineering & Open Source',
    12: 'Scientific Computing & Simulation',
    13: 'Computer Vision & Medical Imaging',
    14: 'Deep Learning & Neural Networks',
}

df_text_abstract['lda_topic'] = [
    get_dominant_topic(bow, lda_abstract) for bow in corpus_abs
]
df_text_corpus['lda_topic'] = [
    get_dominant_topic(bow, lda_corpus) for bow in corpus_corp
]

df_text_abstract['cs_subfield'] = df_text_abstract['lda_topic'].
 ↪map(abstract_topic_labels)
df_text_corpus['cs_subfield']   = df_text_corpus['lda_topic'].
 ↪map(corpus_topic_labels)
```

```
print('Abstract subfield distribution:')
print(df_text_abstract['cs_subfield'].value_counts())
print('\nCorpus subfield distribution:')
print(df_text_corpus['cs_subfield'].value_counts())
```

```
Abstract subfield distribution:
cs_subfield
Computer Vision & Robotics                979
Networking & Wireless                     783
Scientific Computing & Simulation         576
Health & Biomedical AI                    574
Academic Community & Outreach             525
STEM Education & Workforce                510
Algorithms, ML & Quantum                  485
Systems & Architecture                    377
Cybersecurity & Privacy                   372
K-12 CS Education                         305
Autonomous Systems & Transportation       284
Software Engineering & Formal Methods     229
Data Science & Open Source                145
Cyberinfrastructure & HPC                 138
Human-Computer Interaction                116
Name: count, dtype: int64

Corpus subfield distribution:
cs_subfield
Deep Learning & Neural Networks       820
Privacy, Fairness & Social AI         630
Academic Community & Outreach         605
CS Education & STEM                   565
Cybersecurity                         556
Human-Computer Interaction            549
Network Systems & Databases           484
Wireless & Networking                 443
Robotics & Autonomous Systems         374
Scientific Computing & Simulation     337
Algorithms, Quantum & Theory          272
Software Engineering & Open Source    217
Health, Disaster & Resilience         204
Computer Vision & Medical Imaging     177
Systems, Hardware & Edge              165
Name: count, dtype: int64
```

[20]:
```
#   LDA: Awards & funding per subfield per year
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
    subfield_year = (
        df_t.groupby(['year', 'cs_subfield']).agg(
```

```
            num_awards=('awd_id', 'count'),
            total_funding=('amount', 'sum')
        ).reset_index()
    )

    for metric, ylabel, suffix in [
        ('num_awards',   'Number of Awards',  'awards'),
        ('total_funding','Total Funding ($)',  'funding'),
    ]:
        slug = f'{label.lower()}_{suffix}'

        fig_line = px.line(
            subfield_year, x='year', y=metric, color='cs_subfield',
            title=f'NSF CSE {ylabel} per CS Subfield per Year - {label}',
            labels={metric: ylabel, 'year': 'Year', 'cs_subfield': 'CS␣
↪Subfield'},
            markers=True, height=550
        )
        fig_line.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
        save_fig(fig_line, f'19_lda_line_{slug}')
        fig_line.show()

        fig_bar = px.bar(
            subfield_year, x='year', y=metric, color='cs_subfield',
            title=f'NSF CSE {ylabel} by CS Subfield - {label}',
            labels={metric: ylabel, 'year': 'Year', 'cs_subfield': 'CS␣
↪Subfield'},
            barmode='stack', height=550
        )
        fig_bar.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
        save_fig(fig_bar, f'20_lda_stacked_{slug}')
        fig_bar.show()
```

```
Saved → plots/19_lda_line_abstract_awards.png
Saved → plots/20_lda_stacked_abstract_awards.png
Saved → plots/19_lda_line_abstract_funding.png
Saved → plots/20_lda_stacked_abstract_funding.png
Saved → plots/19_lda_line_corpus_awards.png
Saved → plots/20_lda_stacked_corpus_awards.png
Saved → plots/19_lda_line_corpus_funding.png
Saved → plots/20_lda_stacked_corpus_funding.png
```

[21]:
```
#   Total funding by LDA subfield
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
```

```
    topic_funding = (
        df_t.groupby('cs_subfield')['amount']
        .sum().reset_index()
        .sort_values('amount', ascending=False)
    )
    fig = px.bar(
        topic_funding, x='amount', y='cs_subfield', orientation='h',
        title=f'Total NSF Funding by CS Subfield - {label} (2021-2025)',
        labels={'amount': 'Total Funding ($)', 'cs_subfield': 'CS Subfield'},
        color='amount', color_continuous_scale='Blues', height=600
    )
    fig.update_layout(yaxis={'categoryorder': 'total ascending'})
    save_fig(fig, f'21_lda_funding_{label.lower()}')
    fig.show()
```

```
Saved → plots/21_lda_funding_abstract.png
```

```
Saved → plots/21_lda_funding_corpus.png
```

[22]:
```
#   LDA topic distribution area & stacked-bar
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
    topic_year = (
        df_t.groupby(['year', 'cs_subfield'])['awd_id']
        .count().reset_index()
        .rename(columns={'awd_id': 'count'})
    )
    fig_area = px.area(
        topic_year, x='year', y='count', color='cs_subfield',
        title=f'LDA Topic Distribution Across Years - {label}',
        labels={'count': 'Number of Awards', 'year': 'Year', 'cs_subfield': 'CS␣
 ↪Subfield'},
        height=550
    )
    fig_area.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
    save_fig(fig_area, f'22_lda_area_{label.lower()}')
    fig_area.show()
```

```
Saved → plots/22_lda_area_abstract.png
```

```
Saved → plots/22_lda_area_corpus.png
```

1.6.2 6b. BERTopic — Transformer-Based Topic Modeling

[23]:
```
#   Clean text for BERTopic (raw text, not tokens)
def clean_text(text):
    if not isinstance(text, str):
        return ''
    text = re.sub(r'<.*?>', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
```

```
    return text

abstracts_list = df_text_abstract['abstract'].apply(clean_text).tolist()
corpus_list    = df_text_corpus['corpus'].apply(clean_text).tolist()

print(f'Abstract docs: {len(abstracts_list):,}')
print(f'Corpus docs:   {len(corpus_list):,}')
```

```
Abstract docs: 6,398
Corpus docs:   6,398
```

[24]:
```
#   BERT Embeddings
embedding_model = SentenceTransformer('all-MiniLM-L6-v2')

print('Encoding abstracts…')
embeddings_abs = embedding_model.encode(abstracts_list, show_progress_bar=True,␣
 ↪batch_size=64)
print(f'Abstract embeddings shape: {embeddings_abs.shape}')

print('\nEncoding corpus…')
embeddings_corp = embedding_model.encode(corpus_list, show_progress_bar=True,␣
 ↪batch_size=64)
print(f'Corpus embeddings shape: {embeddings_corp.shape}')
```

```
Encoding abstracts…
```

```
Batches:   0%|          | 0/100 [00:00<?, ?it/s]
```

```
Abstract embeddings shape: (6398, 384)

Encoding corpus…
```

```
Batches:   0%|          | 0/100 [00:00<?, ?it/s]
```

```
Corpus embeddings shape: (6398, 384)
```

[25]:
```
#   UMAP + HDBSCAN + BERTopic
umap_model = UMAP(n_neighbors=15, n_components=5, min_dist=0.0,
                  metric='cosine', random_state=42)
hdbscan_model = HDBSCAN(min_cluster_size=30, min_samples=10,
                        metric='euclidean', prediction_data=True)
vectorizer = CountVectorizer(stop_words='english', min_df=5, ngram_range=(1, 2))

topic_model_abstract = BERTopic(
    embedding_model=embedding_model,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer,
    top_n_words=10, verbose=True
)
```

```
topic_model_corpus = BERTopic(
    embedding_model=embedding_model,
    umap_model=umap_model,
    hdbscan_model=hdbscan_model,
    vectorizer_model=vectorizer,
    top_n_words=10, verbose=True
)

print('Fitting BERTopic on abstracts…')
topics_abs, probs_abs = topic_model_abstract.fit_transform(abstracts_list,␣
 ↪embeddings_abs)
df_text_abstract['bert_topic'] = topics_abs
n_topics_abs = len(topic_model_abstract.get_topic_info()) - 1
n_outliers_abs = (df_text_abstract['bert_topic'] == -1).sum()
print(f'BERTopic (abstract): {n_topics_abs} topics, {n_outliers_abs:,} outliers␣
 ↪'
      f'({n_outliers_abs/len(df_text_abstract)*100:.1f}%)')

print('\nFitting BERTopic on corpus…')
topics_corp, probs_corp = topic_model_corpus.fit_transform(corpus_list,␣
 ↪embeddings_corp)
df_text_corpus['bert_topic'] = topics_corp
n_topics_corp = len(topic_model_corpus.get_topic_info()) - 1
n_outliers_corp = (df_text_corpus['bert_topic'] == -1).sum()
print(f'BERTopic (corpus): {n_topics_corp} topics, {n_outliers_corp:,} outliers␣
 ↪'
      f'({n_outliers_corp/len(df_text_corpus)*100:.1f}%)')
```

```
2026-06-02 22:55:23,076 - BERTopic - Dimensionality - Fitting the dimensionality
reduction algorithm

Fitting BERTopic on abstracts…

2026-06-02 22:55:34,536 - BERTopic - Dimensionality - Completed
2026-06-02 22:55:34,537 - BERTopic - Cluster - Start clustering the reduced
embeddings
2026-06-02 22:55:34,607 - BERTopic - Cluster - Completed
2026-06-02 22:55:34,608 - BERTopic - Representation - Fine-tuning topics using
representation models.
2026-06-02 22:55:35,772 - BERTopic - Representation - Completed
2026-06-02 22:55:36,646 - BERTopic - Dimensionality - Fitting the dimensionality
reduction algorithm

BERTopic (abstract): 61 topics, 1,612 outliers (25.2%)


Fitting BERTopic on corpus…

2026-06-02 22:55:41,727 - BERTopic - Dimensionality - Completed
2026-06-02 22:55:41,727 - BERTopic - Cluster - Start clustering the reduced
```

```
embeddings
2026-06-02 22:55:41,796 - BERTopic - Cluster - Completed
2026-06-02 22:55:41,798 - BERTopic - Representation - Fine-tuning topics using
representation models.
2026-06-02 22:55:43,105 - BERTopic - Representation - Completed

BERTopic (corpus): 57 topics, 1,202 outliers (18.8%)
```

[26]:
```python
#   BERTopic top-word bar charts
for label, tm in [('Abstract', topic_model_abstract), ('Corpus',␣
 ↪topic_model_corpus)]:
    print(f'\n  Top Words per BERTopic - {label}  \n')
    info = tm.get_topic_info()
    for _, row in info[info['Topic'] != -1].head(15).iterrows():
        words = [w for w, _ in tm.get_topic(row['Topic'])]
        print(f"T{row['Topic']:3d} ({row['Count']:4,} docs): {' | '.join(words[:
 ↪8])}")

    fig_bc = tm.visualize_barchart(top_n_topics=15, n_words=8)
    fig_bc.update_layout(title=f'BERTopic Top Words - {label}')
    save_fig(fig_bc, f'23_bert_barchart_{label.lower()}')
    fig_bc.show()
```

```
  Top Words per BERTopic - Abstract

T  0 ( 286 docs): wireless | spectrum | networks | network | communication |
communications | radio | 5g
T  1 ( 243 docs): autonomous | safety | systems | learning | control | driving |
vehicles | agents
T  2 ( 214 docs): communities | community | resilience | disaster | food | water
| emergency | civic
T  3 ( 182 docs): models | medical | learning | clinical | ai | data | project |
health
T  4 ( 173 docs): online | social | media | social media | users | information |
news | authentication
T  5 ( 165 docs): memory | computing | performance | systems | hardware |
parallel | project | design
T  6 ( 149 docs): fairness | fair | algorithmic | data | causal | learning |
algorithms | project
T  7 ( 148 docs): network | internet | cloud | performance | dns | applications
| project | networks
T  8 ( 141 docs): cs | teachers | computer science | school | computer |
computing | students | science
T  9 ( 134 docs): quantum | quantum computing | computing | quantum computers |
computers | problems | classical | project
T 10 ( 132 docs): software | code | programming | verification | bug | program |
tools | testing
```

```
T 11 ( 125 docs): security | detection | attacks | attack | malware | techniques
| data | cybersecurity
T 12 ( 122 docs): security | secure | cloud | protocols | cryptographic | mpc |
cryptography | project
T 13 ( 115 docs): conference | students | doctoral | consortium | student |
travel | doctoral consortium | researchers
T 14 ( 112 docs): vr | ar | virtual | reality | immersive | virtual reality |
disabilities | accessibility
  Saved → plots/23_bert_barchart_abstract.png


  Top Words per BERTopic - Corpus

T  0 ( 364 docs): community | communities | civic | resilience | project |
disaster | food | data
T  1 ( 305 docs): edge | network | internet | cloud | applications | project |
computing | performance
T  2 ( 253 docs): memory | hardware | computing | performance | architecture |
project | systems | design
T  3 ( 241 docs): wireless | spectrum | networks | sensing | communication |
communications | radio | 5g
T  4 ( 241 docs): safety | autonomous | learning | driving | vehicles | systems
| control | reinforcement
T  5 ( 153 docs): learning | medical | clinical | data | models | deep | health
| ai
T  6 ( 145 docs): software | code | verification | testing | program | language
| project | bugs
T  7 ( 139 docs): reu | site | reu site | students | research | projects |
undergraduate | program
T  8 ( 128 docs): protein | dna | biological | biology | molecular |
computational | data | project
T  9 ( 128 docs): quantum | quantum computing | computing | classical | quantum
computers | computers | project | problems
T 10 ( 127 docs): graph | graphs | networks | algorithms | project | analysis |
network | gnn
T 11 ( 125 docs): conference | travel | doctoral | students | student |
consortium | international | researchers
T 12 ( 122 docs): language | models | ai | llms | llm | text | languages |
project
T 13 ( 120 docs): cybersecurity | security | attacks | malware | detection |
cyber | research | attack
T 14 ( 110 docs): fairness | fair | algorithmic | learning | decision | social |
bias | machine learning
  Saved → plots/23_bert_barchart_corpus.png
```

[27]:
```python
#   BERTopic funding by topic
for label, df_t, tm in [
    ('Abstract', df_text_abstract, topic_model_abstract),
```

```
    ('Corpus',   df_text_corpus,   topic_model_corpus),
]:
    bt_funding = (
        df_t[df_t['bert_topic'] != -1]
        .groupby('bert_topic')['amount']
        .sum().reset_index()
        .sort_values('amount', ascending=False)
        .head(15)
    )
    bt_funding['topic_label'] = bt_funding['bert_topic'].apply(
        lambda t: f"T{t}: " + tm.get_topic(t)[0][0]
    )
    fig = px.bar(
        bt_funding, x='amount', y='topic_label', orientation='h',
        title=f'Top 15 BERTopics by Total NSF Funding - {label}',
        labels={'amount': 'Total Funding ($)', 'topic_label': 'BERTopic'},
        color='amount', color_continuous_scale='Teal', height=600
    )
    fig.update_layout(yaxis={'categoryorder': 'total ascending'})
    save_fig(fig, f'24_bert_funding_{label.lower()}')
    fig.show()
```

```
Saved → plots/24_bert_funding_abstract.png
```

```
Saved → plots/24_bert_funding_corpus.png
```

[28]:
```
#   BERTopic topics over time (manual)
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
    tm = topic_model_abstract if label == 'Abstract' else topic_model_corpus
    top_topics = (
        df_t[df_t['bert_topic'] != -1]['bert_topic']
        .value_counts().head(10).index.tolist()
    )
    topic_year = (
        df_t[df_t['bert_topic'].isin(top_topics)]
        .groupby(['year', 'bert_topic'])['awd_id']
        .count().reset_index()
        .rename(columns={'awd_id': 'count'})
    )
    topic_year['topic_label'] = topic_year['bert_topic'].apply(
        lambda t: f"T{t}: " + tm.get_topic(t)[0][0]
    )
    fig_line = px.line(
        topic_year, x='year', y='count', color='topic_label',
        title=f'BERTopic Top 10 Topics Over Time - {label}',
        labels={'count': 'Number of Awards', 'year': 'Year', 'topic_label':␣
 ↪'Topic'},
        markers=True, height=550
```

```
    )
    fig_line.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
    save_fig(fig_line, f'25_bert_topics_over_time_{label.lower()}')
    fig_line.show()

    fig_bar = px.bar(
        topic_year, x='year', y='count', color='topic_label',
        title=f'BERTopic Topic Distribution per Year - {label}',
        labels={'count': 'Number of Awards', 'year': 'Year', 'topic_label':␣
↪'Topic'},
        barmode='stack', height=550
    )
    fig_bar.update_xaxes(tickvals=[2021, 2022, 2023, 2024, 2025])
    save_fig(fig_bar, f'26_bert_dist_per_year_{label.lower()}')
    fig_bar.show()
```

```
Saved → plots/25_bert_topics_over_time_abstract.png
Saved → plots/26_bert_dist_per_year_abstract.png
Saved → plots/25_bert_topics_over_time_corpus.png
Saved → plots/26_bert_dist_per_year_corpus.png
```

1.7 7. LDA vs. BERTopic Comparison

[29]:
```
#   Heatmap: LDA vs BERTopic assignment overlap
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
    overlap = df_t[df_t['bert_topic'] != -1].copy()
    overlap['lda_label']  = overlap['cs_subfield']
    overlap['bert_label'] = 'BERT_' + overlap['bert_topic'].astype(str)

    crosstab = pd.crosstab(overlap['lda_label'], overlap['bert_label']).iloc[:,␣
↪:20]

    fig_ht, ax_ht = plt.subplots(figsize=(18, 8))
    sns.heatmap(crosstab, cmap='YlOrRd', linewidths=0.2, annot=False, ax=ax_ht)
    ax_ht.set_title(f'LDA vs BERTopic Assignment Overlap - {label}',
                    fontsize=14, fontweight='bold')
    ax_ht.set_xlabel('BERTopic')
    ax_ht.set_ylabel('LDA CS Subfield')
    plt.xticks(rotation=45, ha='right')
    plt.yticks(rotation=0)
    plt.tight_layout()
    save_fig(fig_ht, f'27_lda_bert_heatmap_{label.lower()}')
    plt.show()
```

```
Saved → plots/27_lda_bert_heatmap_abstract.png
Saved → plots/27_lda_bert_heatmap_corpus.png
```

[30]:
```
#   Abstract vs Corpus subfield distribution
abs_dist  = df_text_abstract['cs_subfield'].value_counts().reset_index()
corp_dist = df_text_corpus['cs_subfield'].value_counts().reset_index()
abs_dist.columns  = ['subfield', 'count']
corp_dist.columns = ['subfield', 'count']
abs_dist['pct']  = abs_dist['count']  / abs_dist['count'].sum()  * 100
corp_dist['pct'] = corp_dist['count'] / corp_dist['count'].sum() * 100

fig_comp, axes = plt.subplots(1, 2, figsize=(18, 7))
axes[0].barh(abs_dist['subfield'],  abs_dist['pct'],  color='steelblue')
axes[0].set_title('Abstract Model - Subfield Distribution (%)',␣
 ↪fontweight='bold')
axes[0].set_xlabel('% of Awards')
axes[0].invert_yaxis()

axes[1].barh(corp_dist['subfield'], corp_dist['pct'], color='coral')
axes[1].set_title('Corpus Model - Subfield Distribution (%)', fontweight='bold')
axes[1].set_xlabel('% of Awards')
axes[1].invert_yaxis()

plt.suptitle('Abstract vs Corpus LDA Model Comparison', fontsize=14,␣
 ↪fontweight='bold')
plt.tight_layout()
save_fig(fig_comp, '28_abstract_vs_corpus_compare')
plt.show()
```

```
Saved → plots/28_abstract_vs_corpus_compare.png
```

[31]:
```
#   Top growing vs declining subfields
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
    subfield_year = (
        df_t.groupby(['year', 'cs_subfield']).agg(
            num_awards=('awd_id', 'count'),
            total_funding=('amount', 'sum')
        ).reset_index()
    )
    for metric, ylabel, slug in [
        ('num_awards',   '% Change in Number of Awards', 'awards'),
        ('total_funding','% Change in Total Funding',    'funding'),
    ]:
        pivot = subfield_year.pivot(
            index='cs_subfield', columns='year', values=metric
        ).fillna(0)
        pivot['pct_change'] = ((pivot[2025] - pivot[2021]) / pivot[2021].
 ↪replace(0, np.nan) * 100).round(1)
        pivot = pivot.dropna(subset=['pct_change']).sort_values('pct_change',␣
 ↪ascending=False)
```

```
        colors = ['green' if x >= 0 else 'red' for x in pivot['pct_change']]

        fig_g, ax_g = plt.subplots(figsize=(12, 7))
        ax_g.barh(pivot.index, pivot['pct_change'], color=colors)
        ax_g.axvline(x=0, color='black', linewidth=0.8, linestyle='--')
        ax_g.set_title(f'CS Subfield Growth 2021-2025 ({ylabel}) - {label}',
                       fontsize=13, fontweight='bold')
        ax_g.set_xlabel(ylabel)
        plt.tight_layout()
        save_fig(fig_g, f'29_growth_{label.lower()}_{slug}')
        plt.show()
```

```
Saved → plots/29_growth_abstract_awards.png
Saved → plots/29_growth_abstract_funding.png
Saved → plots/29_growth_corpus_awards.png
Saved → plots/29_growth_corpus_funding.png
```

[32]:
```
#   Heatmap: subfield × year funding
for label, df_t in [('Abstract', df_text_abstract), ('Corpus', df_text_corpus)]:
    subfield_year = (
        df_t.groupby(['year', 'cs_subfield'])['amount']
        .sum().reset_index()
    )
    pivot = subfield_year.pivot(
        index='cs_subfield', columns='year', values='amount'
    ).fillna(0) / 1e6  # millions

    fig_hm, ax_hm = plt.subplots(figsize=(12, 8))
    sns.heatmap(pivot, cmap='YlOrRd', annot=True, fmt='.1f',
                linewidths=0.3, cbar_kws={'label': 'Funding ($M)'}, ax=ax_hm)
    ax_hm.set_title(f'NSF CSE Funding by CS Subfield and Year ($M) - {label}',
                    fontsize=13, fontweight='bold')
    plt.tight_layout()
    save_fig(fig_hm, f'30_subfield_year_heatmap_{label.lower()}')
    plt.show()
```

```
Saved → plots/30_subfield_year_heatmap_abstract.png
Saved → plots/30_subfield_year_heatmap_corpus.png
```

1.8 8. Save Results

[33]:
```
#   Save enriched CSV datasets
out_cols = ['awd_id', 'awd_titl_txt', 'year', 'amount', 'duration_years',
            'institution', 'state', 'div_abbr', 'pi_name',
            'lda_topic', 'cs_subfield', 'bert_topic']

df_text_abstract[[c for c in out_cols if c in df_text_abstract.columns]].to_csv(
    'nsf_awards_abstract_topics.csv', index=False
```

```
)
df_text_corpus[[c for c in out_cols if c in df_text_corpus.columns]].to_csv(
    'nsf_awards_corpus_topics.csv', index=False
)
print('Saved: nsf_awards_abstract_topics.csv')
print('Saved: nsf_awards_corpus_topics.csv')
```

```
Saved: nsf_awards_abstract_topics.csv
Saved: nsf_awards_corpus_topics.csv
```

[34]:
```
#   Save LDA & BERTopic models
lda_abstract.save('lda_abstract_model')
lda_corpus.save('lda_corpus_model')
print('LDA models saved')

topic_model_abstract.save(
    'bertopic_abstract_model', serialization='safetensors', save_ctfidf=True
)
topic_model_corpus.save(
    'bertopic_corpus_model', serialization='safetensors', save_ctfidf=True
)
print('BERTopic models saved')
```

```
LDA models saved
BERTopic models saved
```

1.9 9. Export All Plots to PDF

[35]:
```
from PIL import Image
from pypdf import PdfWriter, PdfReader
import io

print(f'Plots registered: {len(SAVED_PLOTS)}')
for p in SAVED_PLOTS:
    exists = os.path.exists(p)
    print(f'  {p}  ← {"OK" if exists else "MISSING"}')
```

```
Plots registered: 51
  plots/01_total_funding_per_year.png  ← OK
  plots/02_cfda_funding_trends.png  ← OK
  plots/03_non_cse_cfda_subplots.png  ← OK
  plots/04_cfda_treemap.png  ← OK
  plots/05_div_funding_per_year.png  ← OK
  plots/06_div_awards_per_year.png  ← OK
  plots/07_div_total_funding.png  ← OK
  plots/08_div_total_awards.png  ← OK
  plots/09_div_funding_pie.png  ← OK
  plots/10_inst_total_bar.png  ← OK
  plots/11_inst_treemap.png  ← OK
```

```
plots/12_inst_bubble.png  ← OK
plots/13_state_map_2021.png  ← OK
plots/13_state_map_2022.png  ← OK
plots/13_state_map_2023.png  ← OK
plots/13_state_map_2024.png  ← OK
plots/13_state_map_2025.png  ← OK
plots/14_pi_awards_bar.png  ← OK
plots/15_pi_treemap.png  ← OK
plots/16_pi_bubble.png  ← OK
plots/17_award_duration_hist.png  ← OK
plots/18_lda_coherence.png  ← OK
plots/19_lda_line_abstract_awards.png  ← OK
plots/20_lda_stacked_abstract_awards.png  ← OK
plots/19_lda_line_abstract_funding.png  ← OK
plots/20_lda_stacked_abstract_funding.png  ← OK
plots/19_lda_line_corpus_awards.png  ← OK
plots/20_lda_stacked_corpus_awards.png  ← OK
plots/19_lda_line_corpus_funding.png  ← OK
plots/20_lda_stacked_corpus_funding.png  ← OK
plots/21_lda_funding_abstract.png  ← OK
plots/21_lda_funding_corpus.png  ← OK
plots/22_lda_area_abstract.png  ← OK
plots/22_lda_area_corpus.png  ← OK
plots/23_bert_barchart_abstract.png  ← OK
plots/23_bert_barchart_corpus.png  ← OK
plots/24_bert_funding_abstract.png  ← OK
plots/24_bert_funding_corpus.png  ← OK
plots/25_bert_topics_over_time_abstract.png  ← OK
plots/26_bert_dist_per_year_abstract.png  ← OK
plots/25_bert_topics_over_time_corpus.png  ← OK
plots/26_bert_dist_per_year_corpus.png  ← OK
plots/27_lda_bert_heatmap_abstract.png  ← OK
plots/27_lda_bert_heatmap_corpus.png  ← OK
plots/28_abstract_vs_corpus_compare.png  ← OK
plots/29_growth_abstract_awards.png  ← OK
plots/29_growth_abstract_funding.png  ← OK
plots/29_growth_corpus_awards.png  ← OK
plots/29_growth_corpus_funding.png  ← OK
plots/30_subfield_year_heatmap_abstract.png  ← OK
plots/30_subfield_year_heatmap_corpus.png  ← OK
```

[36]:
```python
#   Convert each PNG → single-page PDF, then merge
# This approach is robust: no LaTeX, no nbconvert, no browser - pure Python.

PDF_OUTPUT = 'NSF_Funding_Analysis_plots.pdf'
writer     = PdfWriter()
skipped    = []
```

```
for png_path in SAVED_PLOTS:
    if not os.path.exists(png_path):
        skipped.append(png_path)
        continue
    try:
        img = Image.open(png_path).convert('RGB')
        page_buf = io.BytesIO()
        # A4 landscape at 150 dpi
        img.save(page_buf, format='PDF', resolution=150)
        page_buf.seek(0)
        reader = PdfReader(page_buf)
        for page in reader.pages:
            writer.add_page(page)
    except Exception as e:
        print(f'  WARNING: could not add {png_path}: {e}')
        skipped.append(png_path)

with open(PDF_OUTPUT, 'wb') as f:
    writer.write(f)

size_mb = os.path.getsize(PDF_OUTPUT) / 1e6
print(f'\nPDF saved → {PDF_OUTPUT}  ({size_mb:.1f} MB, {len(writer.pages)}␣
 ↪pages)')
if skipped:
    print(f'Skipped {len(skipped)} missing files: {skipped}')
```

```
PDF saved → NSF_Funding_Analysis_plots.pdf  (8.3 MB, 51 pages)
```